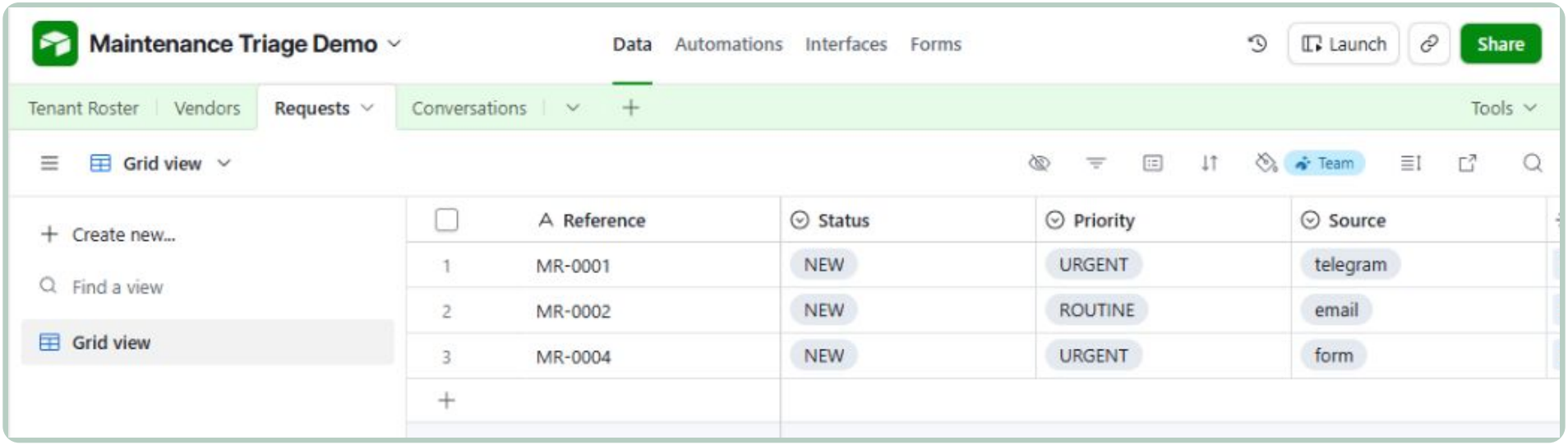
AUTOMATION CASE STUDY

Maintenance request triage

A working local demo that sorts incoming maintenance requests and brings clear, actionable choices to the property manager.



LIVE DEMO OUTPUT / AIRTABLE REQUESTS / SYNTHETIC RECORDS

The problem

Tenant messages arrive through different channels, in different words, without a reliable priority or next step. A manager still has to read, sort and chase details before acting.

What I built

Five connected n8n workflows organize each request, ask for missing detail, surface the right manager decision and record what happened. Vendor contact stays behind a human approval.

SYSTEM DESIGN

One connected decision flow

Email, a web form and a tenant Telegram bot feed one triage process. The manager sees the result in Airtable and approves any vendor email in Telegram.

01 **Tenant Intake**

Email or form -> one standard request -> Triage Engine

02 **Triage Engine**

Read issue -> check urgency and trade -> save ticket

03 **Tenant Telegram**

Collect missing detail or answer a status question

04 **Manager Desk**

Show vendor choices -> manager approves or holds -> record outcome

05 **Operations**

Summaries, unanswered-emergency checks and conversation sync

Behind the screens: n8n routes work; Postgres keeps the ticket and conversation history; Airtable is the manager's visible workspace. The AI suggests an interpretation, while fixed safety rules and manager approval control action.

Two sources, one intake path

Email and web-form requests enter the same triage process.



Actual n8n editor capture - every node visible in the full canvas

TRIGGER A new maintenance email or form submission starts the flow.

STEPS Normalize the message, keep its source context and pass it to the Triage Engine.

DECISION Send one onboarding message when a tenant can continue the request in Telegram.

OUTPUT A saved request, manager handoff and form confirmation or pairing email.

WORKFLOW 02 / TRIAGE ENGINE

Read, check and save

One engine handles the request after any intake channel has normalized it.

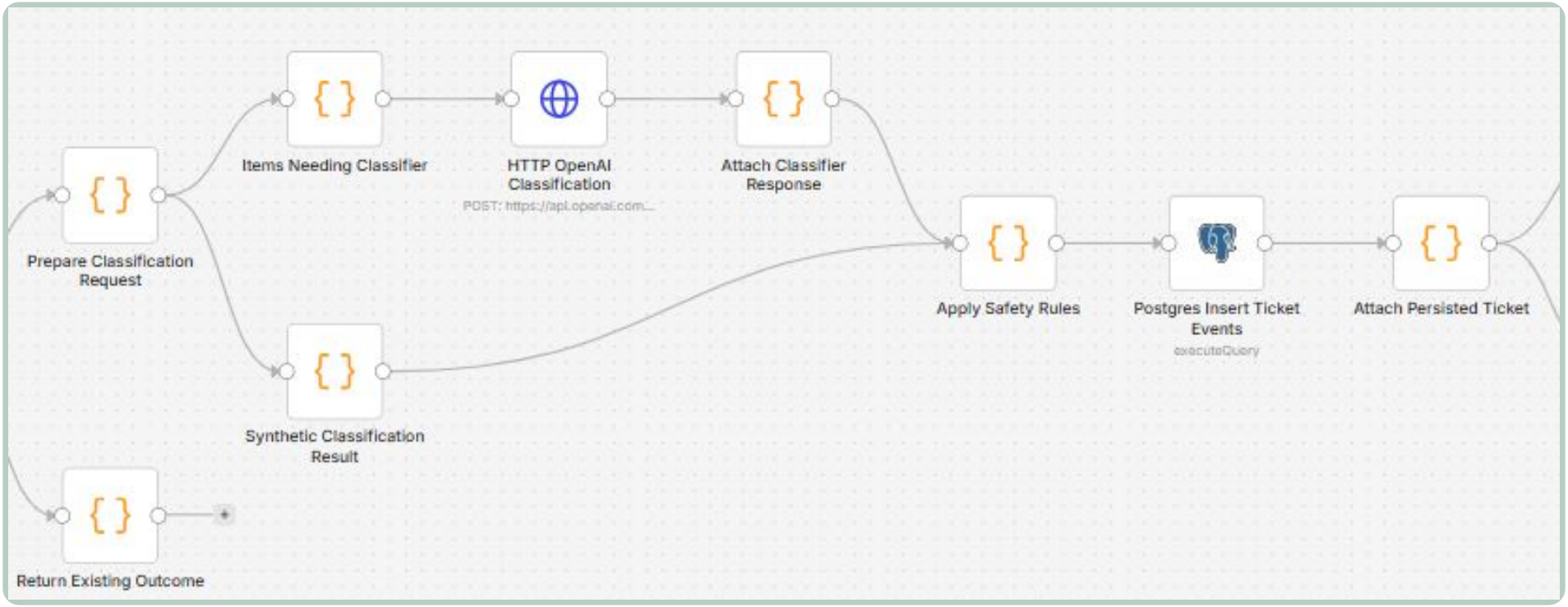


Actual n8n editor capture - every node visible in the full canvas

TRIGGER	Tenant Intake or Tenant Telegram submits a concrete maintenance issue.
STEPS	Check for an existing ticket, classify priority and trade, then save the result.
DECISION	Fixed safety checks can raise urgency; sensitive or unclear cases go to review.
OUTPUT	A ticket with a summary, priority, trade, history and an eligible draft.

The classifier and safety boundary

The model's reading is checked before the ticket becomes a manager decision.

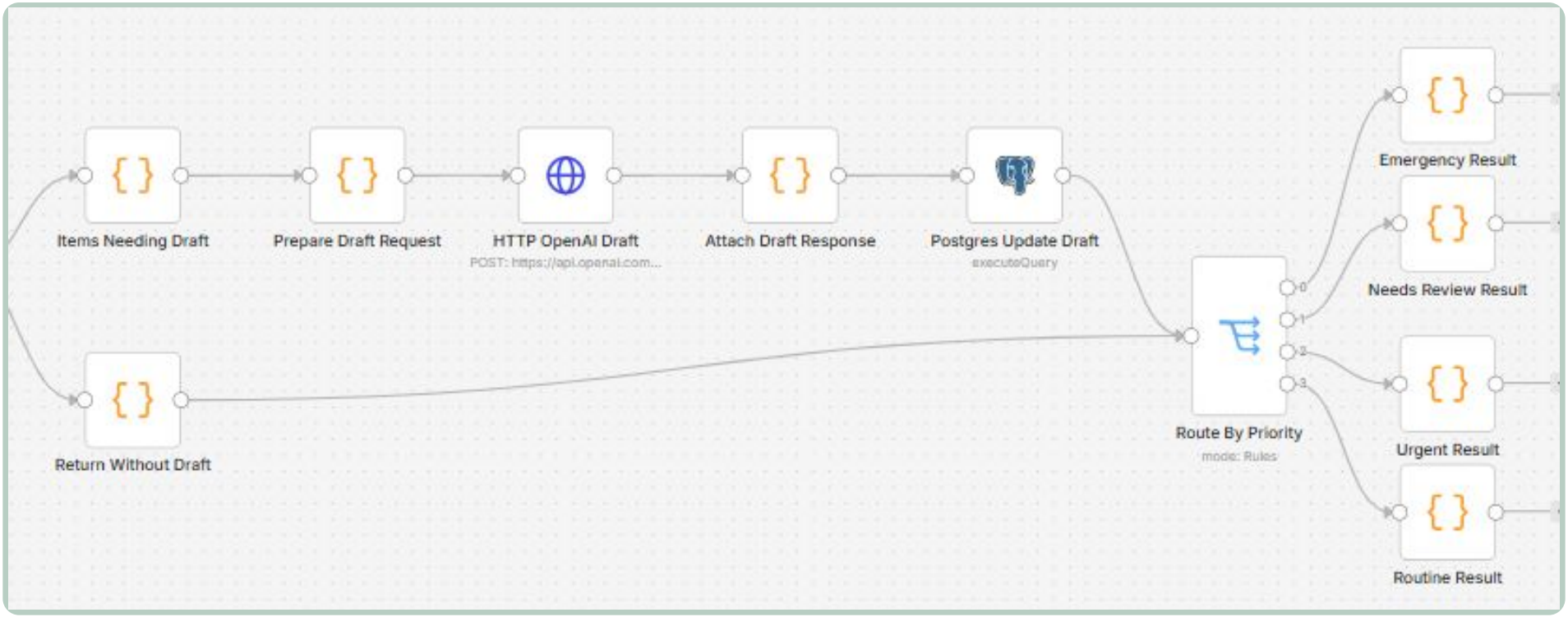


Actual editor crop - classification, safety check and saved request

The AI identifies the issue, priority, trade and missing details. Separate fixed rules then check for safety signals and can raise urgency. The ticket and the reason for its priority are saved before any draft is prepared.

Draft only when allowed

The saved result determines whether a vendor note should be drafted.

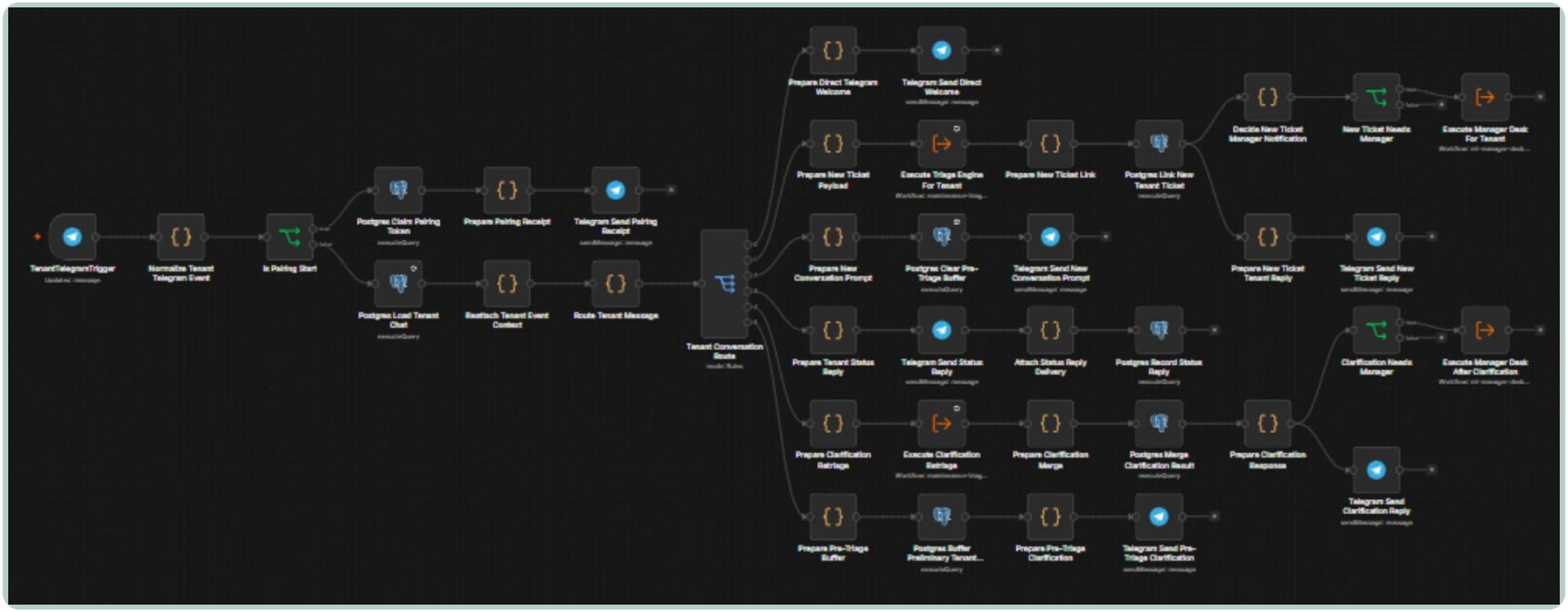


Actual editor crop - draft and output routing

A request that needs human review skips the automatic draft. For eligible maintenance work, the system prepares a vendor note and routes the saved ticket by priority for the next workflow.

Continue the conversation

The tenant bot keeps clarification and status questions tied to the right request.



Actual n8n editor capture - every node visible in the full canvas

TRIGGER A tenant sends a Telegram message or opens a one-time pairing link.

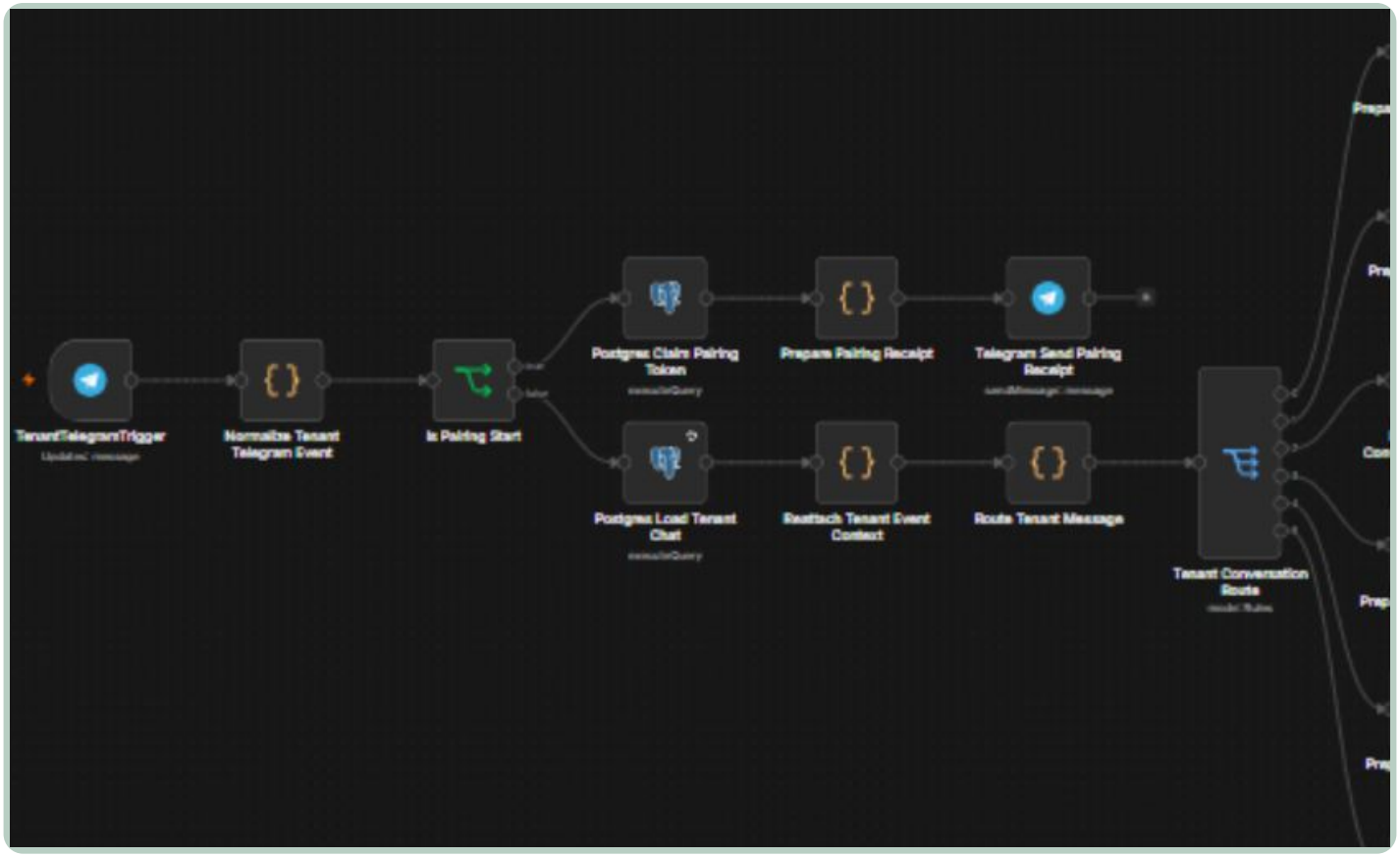
STEPS Load the conversation, recognize the intent and keep each issue in one thread.

DECISION A vague opening prompts for a symptom; a status question does not create a ticket.

OUTPUT A clarification, factual status reply or complete request for manager review.

Pair or route the message

The first decision distinguishes a pairing link from an ordinary tenant message.

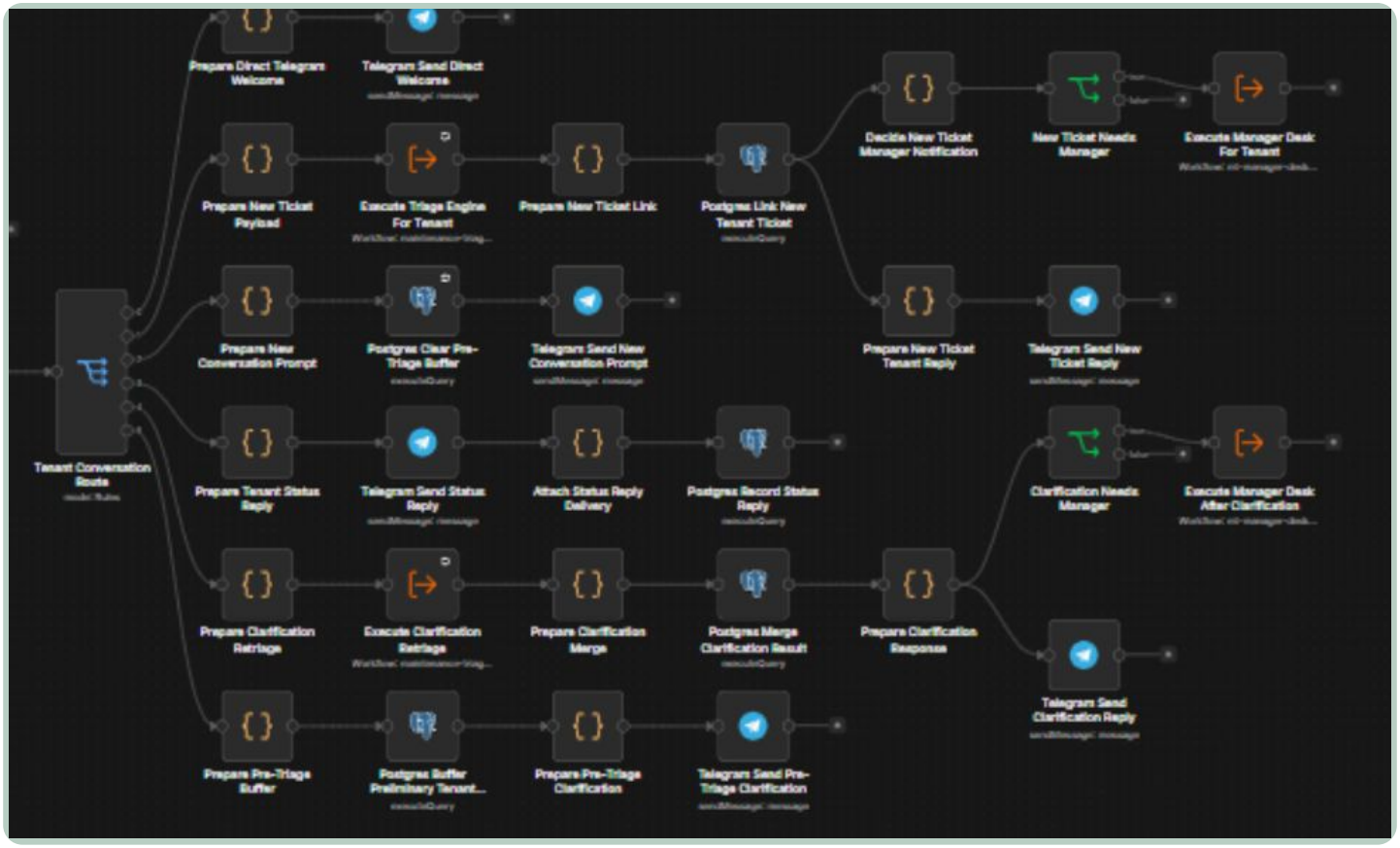


Actual editor crop - pairing and message routing

A pairing link connects the chat to an existing request. Other messages are routed to a welcome, new issue, clarification or status path, using the saved conversation so the tenant does not have to start over.

Keep one issue in one thread

The route for a concrete issue stays separate from status and missing-detail replies.

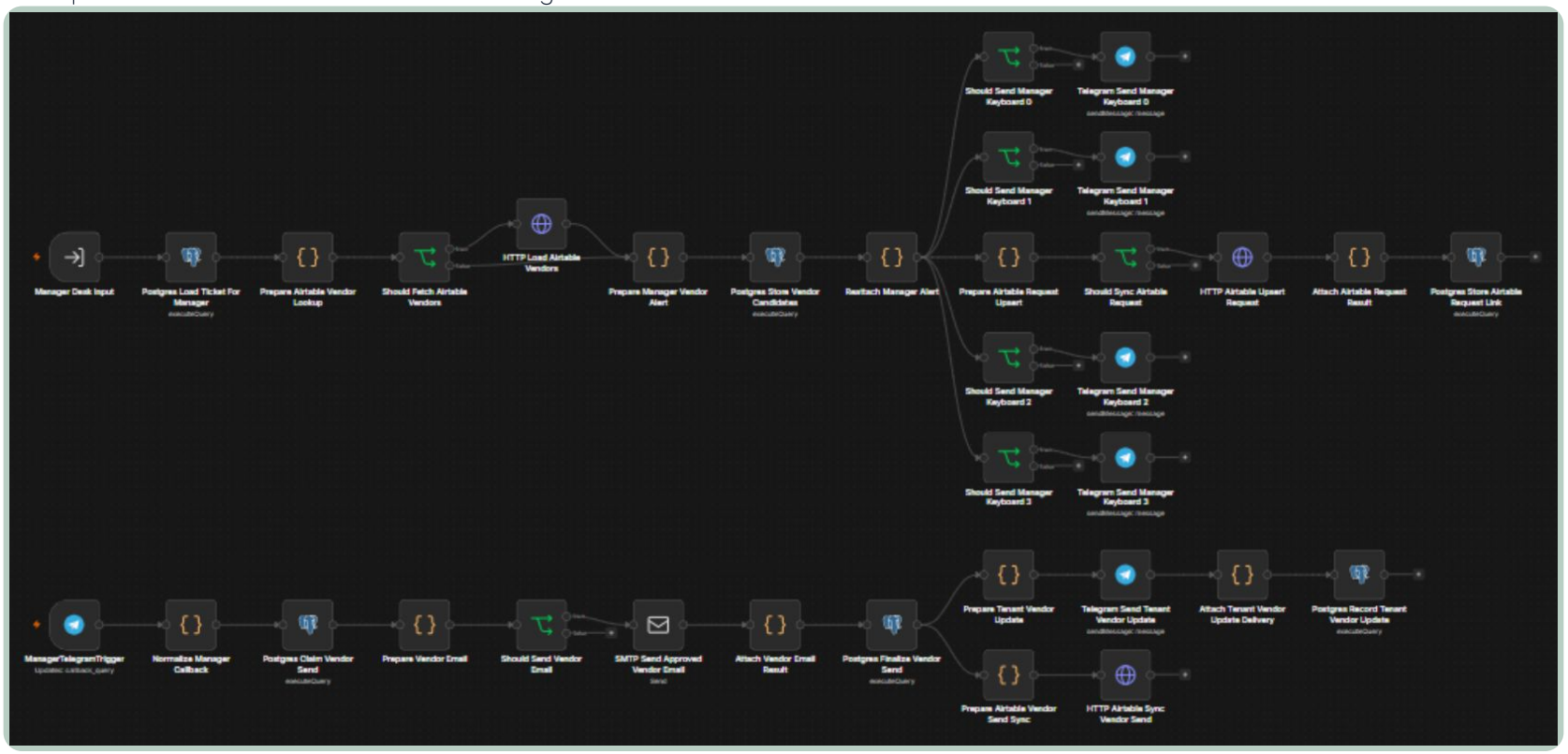


Actual editor crop - new ticket, status and clarification

A concrete symptom becomes one saved request. Missing details prompt targeted questions; a progress question returns the recorded status. When the request is complete, the manager receives it for a decision.

The manager makes the call

A complete request becomes a clear choice of matching vendors or Hold.



Actual n8n editor capture - every node visible in the full canvas

TRIGGER The Triage Engine or Tenant Telegram submits a reviewable request.

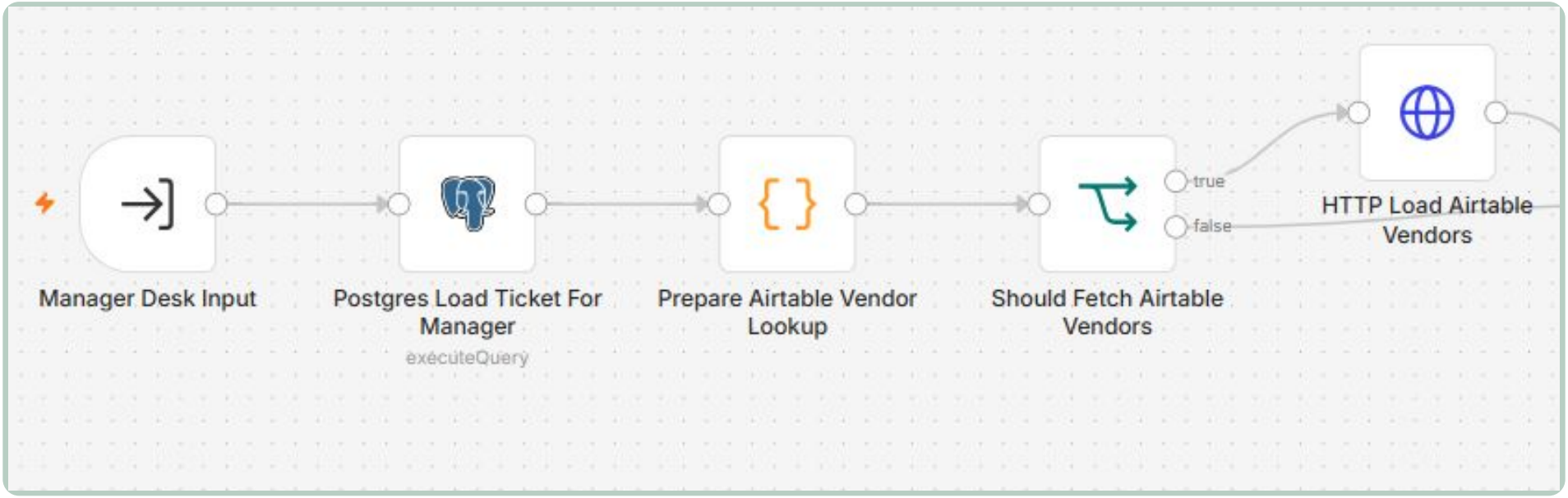
STEPS Match vendors by property and trade; show the summary and draft to the manager.

DECISION The manager selects a vendor or Holds. Review-only cases show no send choice.

OUTPUT One approved vendor email, a recorded result and a tenant update after send success.

Match vendors before alerting

This route prepares a manager message from the saved request and vendor roster.



Actual editor crop - ticket lookup and vendor matching

The flow finds vendors serving the request's property and trade, then presents only those valid options. The request also appears in the manager's Airtable workspace, while the approval remains recorded with the ticket.

One approved vendor email

The lower route starts when the manager taps a vendor button in Telegram.



Actual editor crops - approval, email and recorded outcome

The ticket can claim only one vendor send, so a repeat tap cannot resend it. After the approved email succeeds, the result is saved and the tenant is told a provider was contacted.

Keep the queue visible

Scheduled checks work from saved request and conversation state.



Actual n8n editor capture - every node visible in the full canvas

TRIGGER Time-based schedules start separate summary, alert and sync lanes.

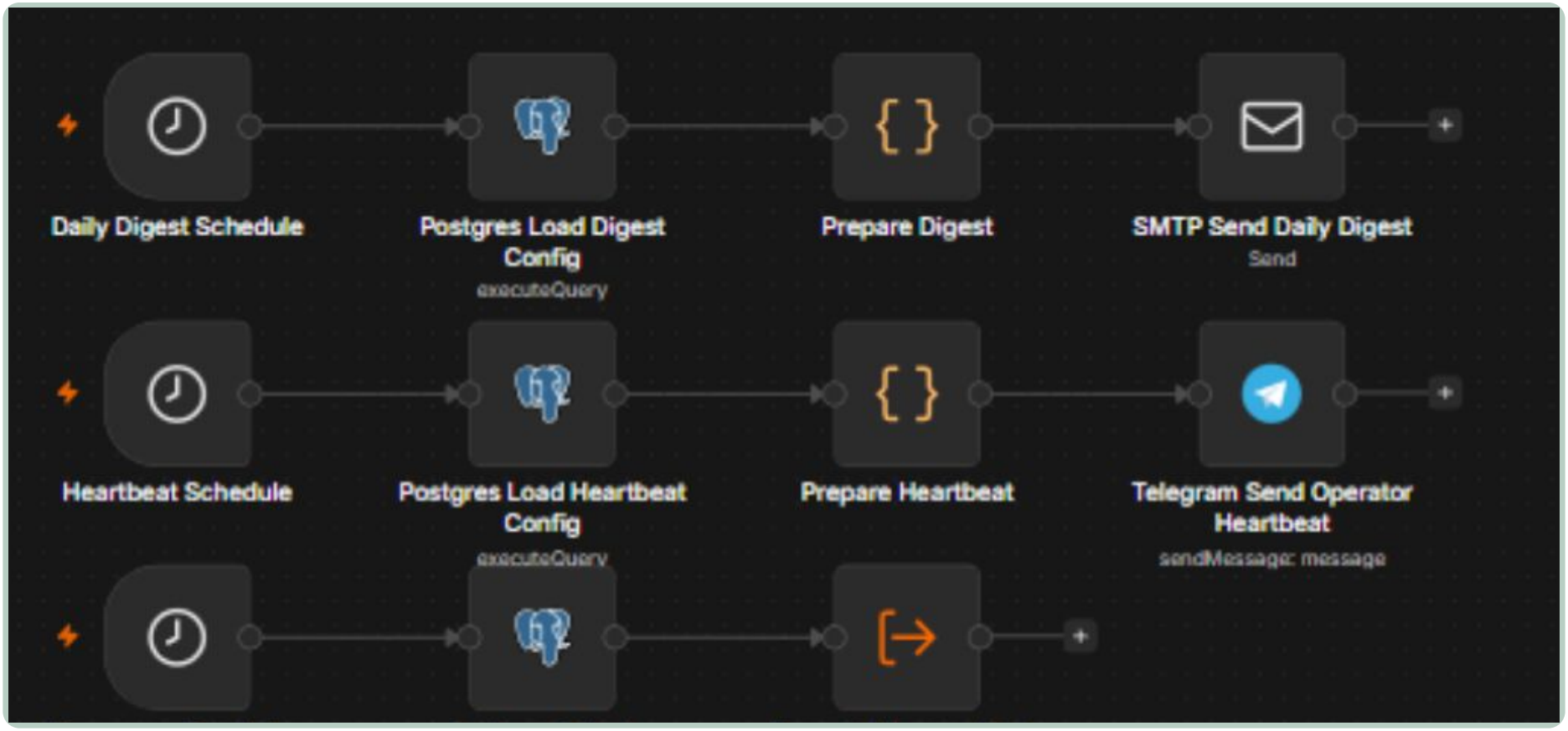
STEPS Read open requests, unanswered emergencies and unsynced conversations.

DECISION Aging marks an item overdue; only an unanswered emergency can re-alert.

OUTPUT A daily summary, operator check or updated conversation view in Airtable.

Summary and alert lanes

The scheduled checks read saved state and run independently of the intake flow.

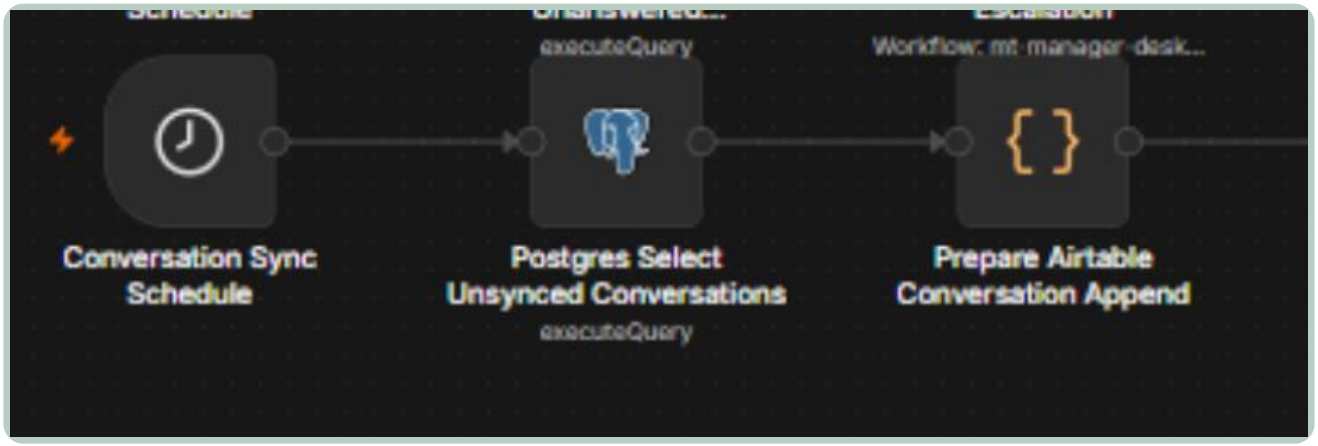


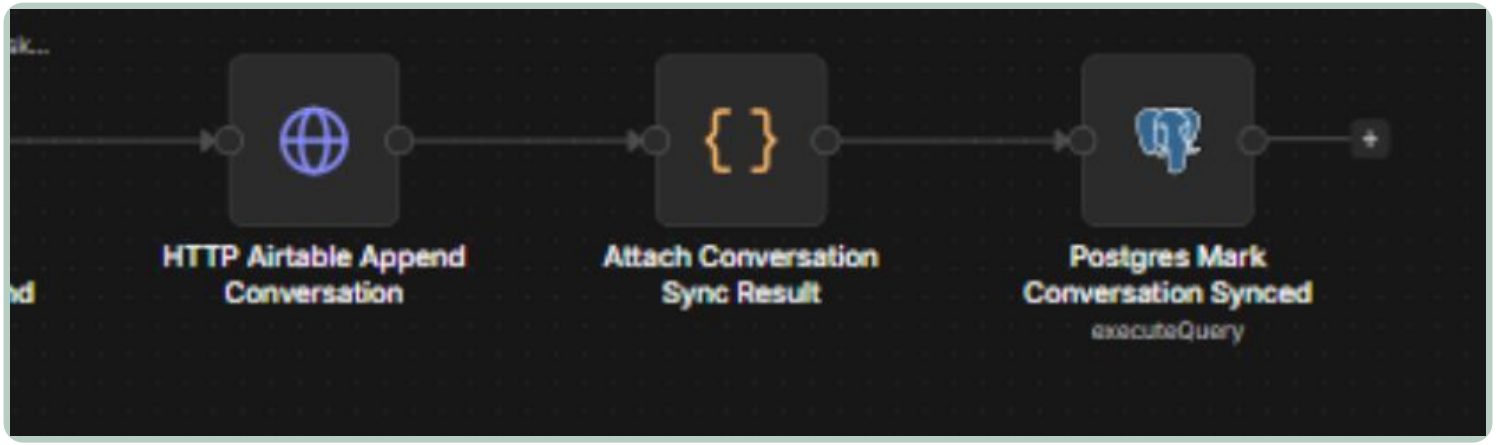
Actual editor crop - digest, operator heartbeat and emergency re-alert

One lane prepares a daily email summary; another checks whether the inbox has gone quiet. The emergency lane looks only for requests that still need a manager response.

Conversation sync

The final lane moves saved conversation entries into the manager's Airtable view.



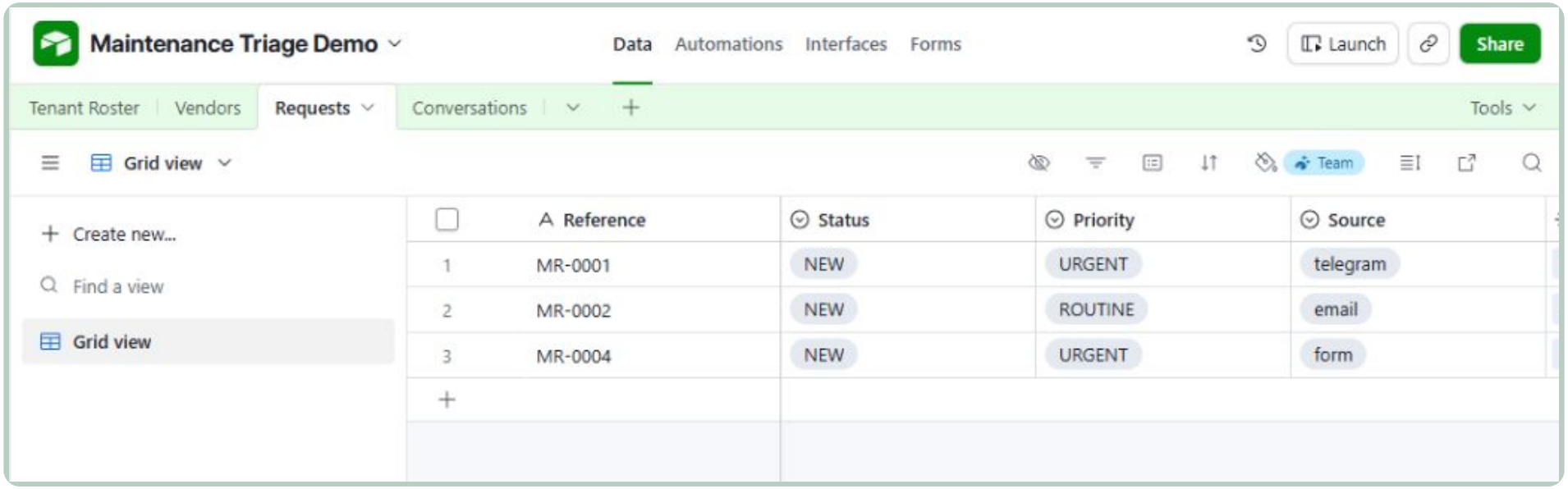


Actual editor crops - select, append and mark synced

The schedule selects unsynced messages, appends them to Airtable and records that the sync completed. This keeps the visible conversation tied to its saved request.

A sortable manager view

The saved request is mirrored into Airtable, where the manager can see its reference, status, priority and source.



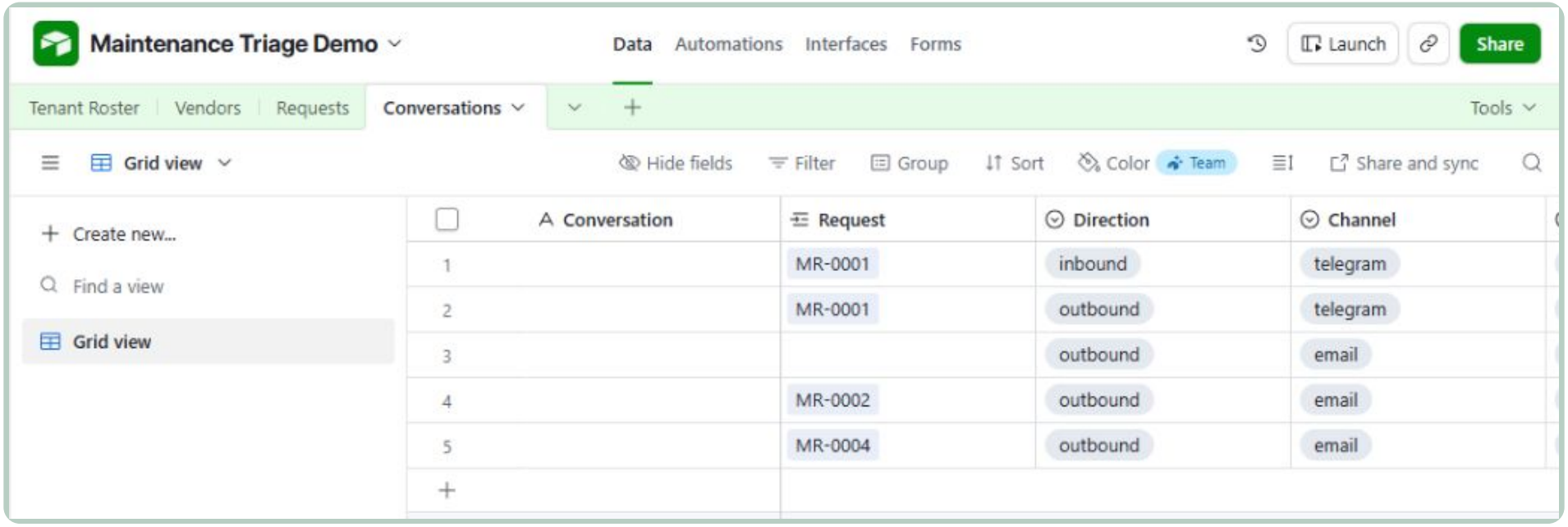
ACTUAL AIRTABLE GRID / SYNTHETIC DEMO REQUESTS

What the visible rows show

The same workspace contains requests that began in Telegram, email and the web form. The priority is saved with each reference, so the manager can scan the queue before opening a conversation.

The follow-up stays attached

A second Airtable view records the request reference, message direction and channel for tenant follow-up.



ACTUAL AIRTABLE GRID / CONTROLLED DEMO CONVERSATIONS

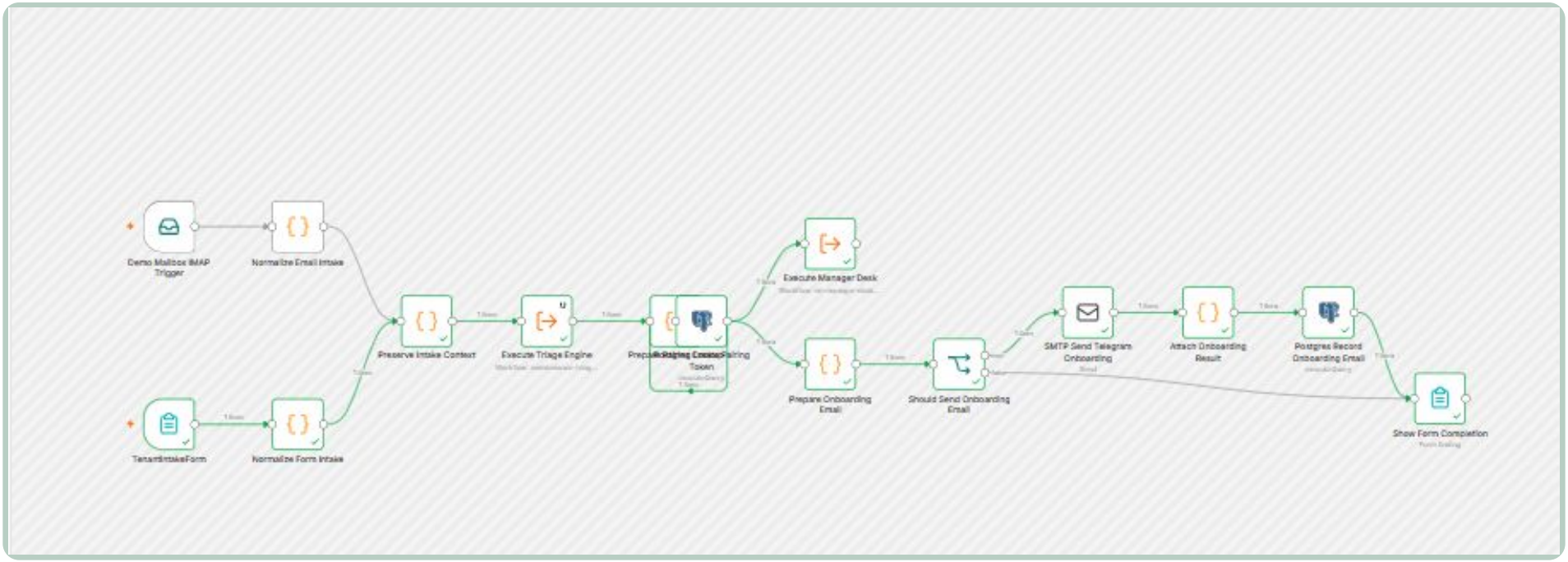
Why this matters

The visible entries include an inbound tenant message and outbound follow-up tied to the same request. Email onboarding and Telegram updates are recorded in the same conversation history.

RECORDED CHECKS

Verified with controlled demo traffic

These checks used synthetic maintenance scenarios and controlled accounts. They describe the demonstrated build, not a client outcome.



ACTUAL N8N EXECUTION / SUCCESSFUL INTAKE PATH

- Six synthetic scenarios were each run five times from clean state: 30/30 expected priorities, with zero Telegram sends in fixture mode.
- A controlled form request completed the intake path and appeared in the Airtable Requests view as an urgent request.
- A live status question produced one factual reply without opening another ticket or invoking the manager decision flow.